Assignment 1: Banking Domain

Topic: Protecting Financial Integrity Through Encapsulation

Scenario:

You are designing a `BankAccount` class for a digital banking system.

The requirements are:

- Account must have:
  - accountNumber (auto-generated, immutable)
  - accountHolderName
  - balance
- User should be able to:
  - deposit money
  - withdraw money
  - check balance
- Withdrawal should not allow:
  - negative withdrawal amount
  - balance going below zero
- Interest calculation should be internal logic and should NOT be callable from outside.
- Account balance must NEVER be directly modified from outside.

Assignment Tasks:

1. Design the `BankAccount` class using proper access modifiers.
2. Decide which fields must be:
   - private
   - public
   - final
3. Implement business rule protection using encapsulation.
4. Explain:
   - Why `balance` cannot be public.
   - What could go wrong if deposit/withdraw logic is bypassed.
5. Add validation logic so that account state cannot become invalid.
6. Use first principle reasoning:
   - What invariant must always remain true?
   - How does encapsulation guarantee this?

Required Thinking:

Reason about:

- Data hiding
- Security
- Financial invariants
- Controlled state transition
- Preventing misuse

Assignment 2: Insurance Claim Processing System

Topic: Encapsulation and Business Rule Enforcement

Scenario:

You are building a `InsuranceClaim` class.

Each claim has:

- claimId (immutable)
- policyNumber
- claimAmount
- claimStatus (Filed, Approved, Rejected, Settled)
- approvedAmount

Business Rules:

- claimAmount cannot be negative.
- approvedAmount cannot exceed claimAmount.
- claimStatus cannot jump directly from "Filed" to "Settled".
- Once claim is "Settled", it cannot be modified.
- Status change must follow logical order:
  Filed → Approved/Rejected → Settled

Assignment Tasks:

1. Design the class ensuring illegal state transitions are impossible.
2. Ensure no external class can directly change claimStatus.
3. Use encapsulation to enforce state transition rules.
4. Explain:
   - Why setter methods alone are not safe.
   - Why status should not be public.
5. Add methods:
   - approveClaim()
   - rejectClaim()
   - settleClaim()
     that enforce valid transitions.
6. Explain:
   - What core invariant must never break?
   - How encapsulation protects system integrity?

Required Thinking:

Reason about:

- Controlled state machine
- Defensive programming
- Business rule constraints
- Immutable IDs
- Preventing invalid transitions

Assignment 3: E-Commerce Inventory System

Topic: Encapsulation and Data Integrity

Scenario:

You are designing a `Product` class for inventory tracking.

Product must have:

- productId (immutable)
- productName
- price
- stockQuantity

Business Rules:

- price cannot be negative
- stock cannot be negative
- stock reduction allowed only through order processing
- price update allowed only by admin
- Once product is discontinued, stock cannot be modified

Assignment Tasks:

1. Design class using proper access modifiers.
2. Prevent direct stock manipulation from outside.
3. Create:
    - increaseStock()
    - reduceStock()
      with validation.
4. Add method discontinueProduct() and prevent further stock updates.
5. Explain:
    - Why exposing stockQuantity publicly is dangerous.
    - How encapsulation prevents inconsistent inventory.
6. Identify:
    - Which fields should be final?
    - Which methods should be private?
    - Which operations must be validated?
7. From first principle:
    - What invariant defines a valid product state?
    - How does encapsulation enforce lifecycle control?